CH6: Common Mistakes in Declarations and Definitions

1. Confusing pointer-field initialization with multidimensional array initialization

Students often assume pointer fields can be initialized like 2D arrays. However, pointer arrays may only store addresses, not raw data blocks.

Wrong:

```
WORD *p[] = { {1,2,3}, {4,5,6} };   // incorrect: nested data blocks
```

Correct:

```
WORD a1[] = {1,2,3};
WORD a2[] = {4,5,6};
WORD *p[] = { a1, a2 };             // correct: addresses only
```

NOTE: A pointer array is not a multidimensional array. It only stores pointers.

2. Assuming pointer reassignment modifies the original string

Beginners often think changing a pointer's target will update the old string.
In reality, the pointer simply stops pointing to the old data.

Example:

```
p_ctl[1] = "New Text";
```

After this instruction, the string "SOH" is lost and unreachable.

NOTE: This is different from 2D char arrays, where data stays fixed in memory.

3. Trying to modify string literals through pointer fields

Literal strings like "NUL", "SOH" reside in read-only memory. Writing into them leads to undefined behavior or program crash.

Wrong:

```
p_ctl[1][0] = 'X';  // wrong: literal is read-only
```

Correct:

```
char soh[] = "SOH";
p_ctl[1] = soh;     // correct: now modifiable
```

NOTE: Only arrays like char name[] can be safely modified.

4. Assuming pointer-based string tables are "rectangular"

Students coming from char msg[10][20] expect all strings in pointer arrays to have equal length. This is false.

Example:

```
char *msg[] = { "A", "Test", "Longer Message" };
```

NOTE: Pointer tables allow variable-length strings. this is a key benefit.

5. Treating rows of a 2D char array as replaceable objects

A full row in a 2D char array cannot be replaced with a new literal.

Wrong:

```
a_ctl[1] = "New";   // invalid in C
```

Correct:

```
strcpy(a_ctl[1], "New"); // overwrite character-by-character
```

NOTE: Arrays are fixed memory blocks; they cannot be reassigned like pointers.

6. Misunderstanding pointer arithmetic on top-level pointer fields

It's wrongly assumable:

"p = p + 3 moves to the next language table."

Example:

```
p = d;
p = p + 3;   // programmer expects p → e
```

This is **undefined behavior**. Pointer arithmetic works only inside the same field.

NOTE: The layout of pointer fields in memory is not guaranteed or sequential.

7. Using incorrect dereference depth in multi-level pointer structures

In the multilingual table, p has type BYTE ***.
It's often dereferenced at the wrong depth.

Wrong:

```
p[1][3] = "Text";      // wrong: p[1][3] is BYTE*, expecting char*
```

Correct:

```
p[1][3]       // pointer to string (BYTE*)
p[1][3][2]    // the 3rd character of that string
```

NOTE: Each level of * corresponds exactly to the number of brackets.

8. Dereferencing pointer fields before assigning them

Unassigned pointer-to-pointer variables are extremely dangerous.

Wrong:

```
BYTE ***p;
p[1][2][3];  // undefined behavior
```

Correct:

```
p = d;       // assign the language table first
p[1][2][3];
```

NOTE: Multi-level pointers must always be initialized before dereferencing.

9. Assuming pointer string literals are stored contiguously

```
BYTE *p_ctl[] = { "NUL", "SOH", "STX" };
```

A common mistake is to think that strings are side by side in memory.

In fact, each constant can be stored separately, even in different segments.

NOTE: Literal storage and pointer-table storage are independent.

10. Believing pointer replacement performs a "deep update"

Changing one pointer in a multi-level table does not modify other structures.

Wrong expectation:

```
p_ctl = newTable;   // student expects all messages to update
```

Correct understanding:

Only the address that the pointer refers to changes; the underlying data stays exactly the same.